

Frankenstein NLP Project

Anders Villwock

2025-11-24

Table of contents

Introduction	1
Setup: Libraries, Functions, and Data Loading	2
Libraries	2
Functions	2
Sourcing Raw Text	2
Initial EDA	3
Cleaning + Tokenizing Text	3
Cleaning	3
Tokenizing	4
for 1831 text	4
For 1818 Text	5
Unifying Dataframes	5
First Visual Analysis	6
Solution 1: Randomly Dropping Values in 1831	6
Solution 2: Imputing new values in 1818	7
All Graphs	8
Discussion	9

Introduction

The Frankenstein NLP project is an independent project that is intended to explore the usage of the Natural Language Processing and line graphs as a visual narrative analysis between two editions of the same book, the 1818 edition and the 1831 edition. The goal of this notebook is to measure quantitatively how different two version of the same story can be with an nlp sentiment perspective. This notebook will document the setup, eda, solutions to certain complications, visualizations, and a discussion of results.

Setup: Libraries, Functions, and Data Loading

Libraries

```
# Standard Imports
#%pip install nltk

import pandas as pd
import nltk
from nltk.sentiment.vader import SentimentIntensityAnalyzer
from nltk.tokenize import sent_tokenize

import requests

import matplotlib.pyplot as plt
import seaborn as sns
import re

# Only required on the first run
#nltk.download('vader_lexicon')
#nltk.download('punkt', force=True)
#nltk.download('punkt_tab')
```

Functions

```
import re

def cut_at_anchor(raw_text, anchor):
    # remove HTML first
    clean = re.sub(r'<[^>]+>', '', raw_text)

    # normalize whitespace
    clean = re.sub(r'\s+', ' ', clean)

    # search for chapter heading
    m = re.search(re.escape(anchor), clean)
    if m:
        return clean[m.start():]

    print("Anchor not found!")
    return clean

# Instantiate our analyzer
sid = SentimentIntensityAnalyzer()
```

Sourcing Raw Text

```
frank31 = requests.get('https://www.gutenberg.org/cache/epub/84/pg84-images.html').text
```

```
frank18 = requests.get('https://www.gutenberg.org/files/41445/41445-h/41445-h.htm').text
```

Initial EDA

```
len(frak31)
```

```
465855
```

```
len(frak18)
```

```
430075
```

At this point, we need to consider some additional cleaning steps. The text retrieved from Project Gutenberg contains a lot of extraneous information, such as the Project Gutenberg header and footer, which we need to remove before performing sentiment analysis. We also have other artifacts such as `//r//n` line breaks, `'\r\n`

```
\r\n\r\n
```

```
\r\n
```

```
Frankenstein 1831 EDA
```

```
-----
```

```
Carriage returns (\r): 9010
```

```
Newlines (\n): 9010
```

```
<p> or </p> tags: 1499
```

```
Frankenstein 1818 EDA
```

```
-----
```

```
Carriage returns (\r): 7378
```

```
Newlines (\n): 7378
```

```
<p> or </p> tags: 1526
```

Cleaning + Tokenizing Text

Cleaning

```
# Remove carriage returns and newlines
frank31_clean = re.sub(r'[\r\n]+' , ' ', frank31)
frank18_clean = re.sub(r'[\r\n]+' , ' ', frank18)

# Optionally remove HTML paragraph artifacts
frank31_clean = re.sub(r'</?p>' , ' ', frank31_clean)
frank18_clean = re.sub(r'</?p>' , ' ', frank18_clean)

# Print a short preview and some basic info to avoid dumping the entire HTML
#print("frank31_clean preview:", frank31_clean[:500].strip())
print("frank31_clean length:", len(frank31_clean))
print("frank18_clean length:", len(frank18_clean))
```

```
frank31_clean length: 450686
```

```
frank18_clean length: 416349
```

```
# This gets rid of the html docstring
frank31_clean = re.sub(r'<[^>]+>' , ' ', frank31_clean),
frank18_clean = re.sub(r'<[^>]+>' , ' ', frank18_clean),
```

```
frank18_clean = cut_at_anchor(frank18, "I am by birth a Genevese; and my family is one of the most distinguished
frank31_clean = cut_at_anchor(frank31, " and my family is one of the most distinguished of that republic
#for currently unknown reasons, "I am by birth..." does not get recognized. I will for now continue as
```

Tokenizing

For this section, we are tokenizing the text so that they may be recognized by the sentiment analysis.

for 1831 text

```
sentence_lev_sent_31 = []

for s in sent_tokenize(frak31_clean):
    sentence_lev_sent_31.append(sid.polarity_scores(s)['compound'])

sentence_lev_df_31 = pd.DataFrame(
    {'Sentence_31': sent_tokenize(frak31_clean),
     'Sentiment_31': sentence_lev_sent_31}
)

sentence_lev_df_31['Sentiment_Smooth31'] = sentence_lev_df_31['Sentiment_31'].rolling(30).mean()

sentence_lev_df_31.sample(5)
```

	Sentence_31	Sentiment_31	Sentiment_Smooth31
362	I had worked hard for nearly two years, for th...	-0.1027	0.111817
874	Frankenstein, your son, your kinsman, your ear...	0.7202	-0.091617
776	Elizabeth saw even this last resource, her exc...	-0.7717	-0.283520
1643	She continued her course along the precipitous...	0.0000	0.030713
2287	But this is false reasoning.	0.0000	0.016933

```
sentence_lev_df_31['Sentiment_Smooth31'].isnull().sum()
```

```
np.int64(29)
```

The null values can be explained due to the value chosen for rolling mean which is 30, so the first 29 values will be NaN.

```
sentence_lev_df_31['Sentence_31']

0      and my family is one of the most distinguishe...
1      My ancestors had been for many years counsellor...
2      He was respected by all who knew him for his i...
3      He passed his younger days perpetually occupie...
4      As the circumstances of his marriage illustrat...
...
2975    For forty years, he produced and distributed P...
2976    Project Gutenberg eBooks are often created fro...
2977    Thus, we do not necessarily keep eBooks in com...
2978    Most people start at our website which has the...
2979    This website includes information about Projec...
Name: Sentence_31, Length: 2980, dtype: object
```

For 1818 Text

```
sentence_lev_sent_18 = []

for s in sent_tokenize(frank18_clean):
    sentence_lev_sent_18.append(sid.polarity_scores(s)['compound'])

sentence_lev_df_18 = pd.DataFrame(
    {'Sentence_18': sent_tokenize(frank18_clean),
     'Sentiment_18': sentence_lev_sent_18}
)

sentence_lev_df_18['Sentiment_Smooth18'] = sentence_lev_df_18['Sentiment_18'].rolling(30).mean()

sentence_lev_df_18.sample(5)
```

	Sentence_18	Sentiment_18	Sentiment_Smooth18
218	Two years passed in this manner, during which ...	0.7469	0.350503
1055	When night came again, I found, with pleasure,...	0.7430	0.138573
1225	Her hair of a shining raven black, and curious...	0.9186	0.430290
65	I feel pleasure in dwelling on the recollectio...	0.8481	0.502900
1240	I ardently desired to understand them, and ben...	0.1406	0.435617

```
sentence_lev_df_18['Sentiment_Smooth18'].isnull().sum()
```

```
np.int64(29)
```

Unifying Dataframes

```
df = sentence_lev_df_18.merge(
    sentence_lev_df_31,
    left_index=True,
    right_index=True,
    how='outer',
    suffixes=('_18', '_31')
)
print("df shape:", df.shape)
```

#Note: that this step is arbitrary as there isn't any mappable meaning to matching the indexes across b
#However, this does not hinder the analysis, so this portion will be kept and potentially updated in the

```
df shape: (2980, 6)
```

```
df.sample(5)
```

	Sentence_18	Sentiment_18	Sentiment_Smooth18	Sentence_31
469	Their melancholy is soothing, and their joy el...	0.4939	0.096380	A servant in Geneva
1469	This, I thought, was the moment of decision, w...	0.0000	0.305117	I often referred the s
1399	â€œAnother circumstance strengthened and confi...	0.4215	0.237243	They conversed with
2955	NaN	NaN	NaN	Section 3.
1240	I ardently desired to understand them, and ben...	0.1406	0.435617	I distinguished severa

```
df.head(10) # change this value to > 35 to see changes in sentiment smooth variables
```

	Sentence_18	Sentiment_18	Sentiment_Smooth18	Sentence_31
0	I am by birth a Genevese; and my family is one...	0.0000	NaN	and my family is one of t
1	My ancestors had been for many years counsello...	0.5719	NaN	My ancestors had been f
2	He was respected by all who knew him for his i...	0.6908	NaN	He was respected by all v
3	He passed his younger days perpetually occupie...	0.3612	NaN	He passed his younger da
4	As the circumstances of his marriage illustrat...	0.0000	NaN	As the circumstances of l
5	One of his most intimate friends was a merchan...	0.0202	NaN	One of his most intimate
6	This man, whose name was Beaufort, was of a pr...	0.4939	NaN	This man, whose name w
7	Having paid his debts, therefore, in the most ...	0.5256	NaN	Having paid his debts, th
8	My father loved Beaufort with the truest frien...	0.6378	NaN	My father loved Beaufort
9	He grieved also for the loss of his society, a...	-0.2500	NaN	He bitterly deplored the

By this point, we have the rolling mean sentiment scores for both versions of Frankenstein. As we can see, by index value 9, the datasets diverge.

First Visual Analysis

```
rough_graph = plt.figure(figsize=(30, 4))
sns.lineplot(x=df.index.to_list(), y='Sentiment_Smooth31', data=df, label = 'Frankenstein 1831')
sns.lineplot(x=df.index.to_list(), y='Sentiment_Smooth18', data=df, label = 'Frankenstein 1818')
```

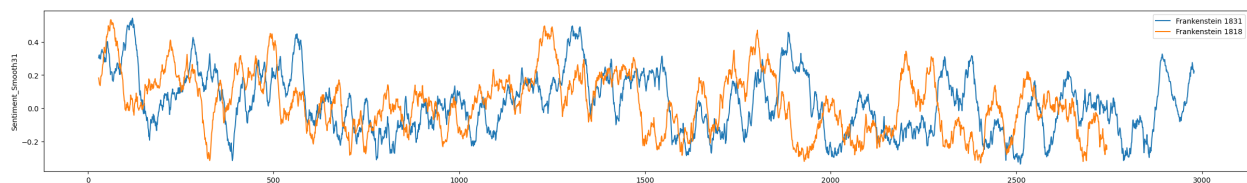


Figure 1: A line plot indicating the sentiment scores of two Frankenstein editions over the course of the book by sentence length.

```
print(f"The overall difference in book length is {len(frunk18_clean) - len(frunk31_clean)}")
```

The overall difference in book length is -27559

Notice how 1831 version is longer than the 1818 version. This is very much the expected outcome of comparing two different books. In order to solve this issue, we can impute or randomly drop values from either the minority or majority class variable respectively. We will do both methods and compare for reliability.

Solution 1: Randomly Dropping Values in 1831

```
df.sample(3)
```

	Sentence_18	Sentiment_18	Sentiment_Smooth18	Sentence_31
315	no mortal could support the horror of that cou...	-0.4939	-0.187623	I prepared myself for a
721	The tortures of the accused did not equal mine...	-0.5719	-0.154543	My tale was not one to
2201	balanced by those horrors of remorse and guilt...	-0.8957	0.288727	More than once the agi

```
df_dropped = pd.DataFrame(
    df['Sentiment_Smooth31']
)
```

```
df_temp_18 = pd.DataFrame(
    df['Sentiment_Smooth18']
).dropna()
```

```
len(df_temp_18)
```

```
2716
```

```
len(df_dropped)
```

```
2980
```

```
df_dropped = df_dropped.sample(n=df_temp_18.shape[0]).sort_index().reset_index(drop=True)
```

```
len(df_dropped)
```

```
2716
```

```
df_dropped.shape
```

```
(2716, 1)
```

```
print(
    len(df['Sentence_18']),
    df['Sentence_18'].isnull().sum()
)
```

```
2980 235
```

```
print(
    len(df['Sentence_31']),
    df['Sentence_31'].isnull().sum()
)
```

```
2980 0
```

```
dropped_graph = plt.figure(figsize=(30, 4))
sns.lineplot(x=df_dropped.index.to_list(), y='Sentiment_Smooth31', data=df_dropped, label = 'Frankenstein 1831')
sns.lineplot(x=df.index.to_list(), y='Sentiment_Smooth18', data=df, label = 'Frankenstein 1818')
```

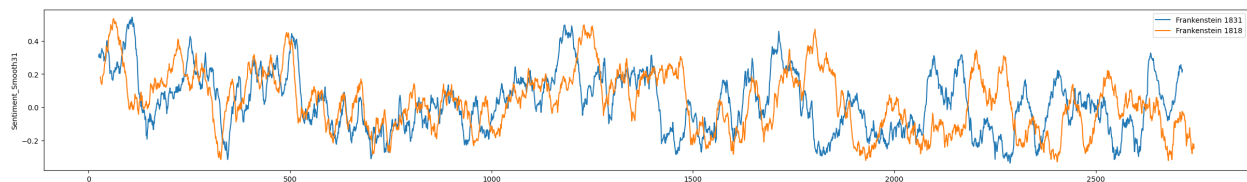


Figure 2: A line plot indicating the sentiment scores of two Frankenstein editions over the course of the book by sentence length.

Solution 2: Imputing new values in 1818

The method to perform this operation will be to get a scale factor by which to increase the length of the 1818 text to. This will be done by dividing the length of the 1831 text by the length of the 1818 text.

The length of the 1831 edition is 2980 and the length of the 1818 edition is 2745

```
scale_factor = 2980 / 2745
scale_factor
```

```
1.0856102003642987
```

```
df_impute = pd.DataFrame(
    df['Sentiment_Smooth18']
)
```

```
new_index = df_impute.index.to_list()
```

```
new_index = [int(i * scale_factor) for i in new_index]
```

```
df_impute.index = new_index
df_impute
```

	Sentiment_Smooth18
0	NaN
1	NaN
2	NaN
3	NaN
4	NaN
...	...
3229	NaN
3230	NaN
3231	NaN
3232	NaN
3234	NaN

```
imputed_graph = plt.figure(figsize=(30, 4))
sns.lineplot(x=df.index.to_list(), y='Sentiment_Smooth31', data=df, label = 'Frankenstein 1831')
sns.lineplot(x=df_impute.index.to_list(), y='Sentiment_Smooth18', data=df_impute, label = 'Frankenstein
```

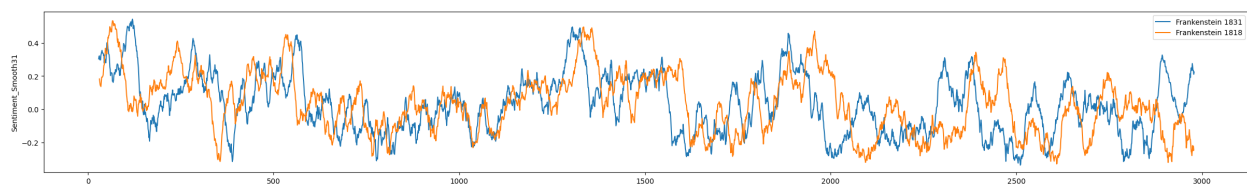
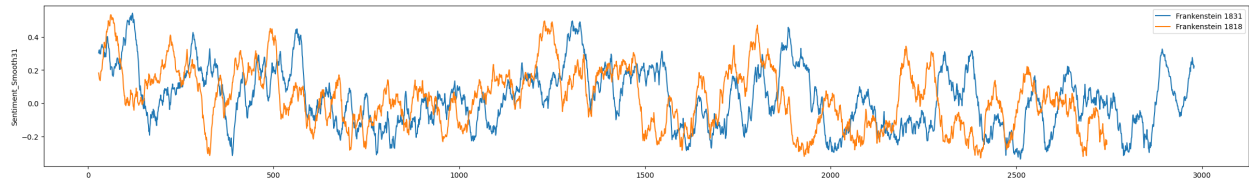


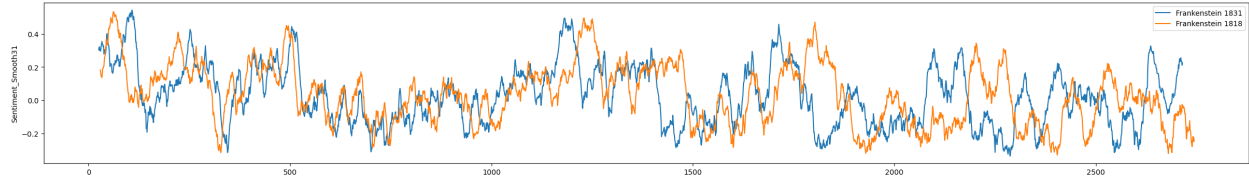
Figure 3: A line plot indicating the sentiment scores of two Frankenstein editions over the course of the book by sentence length.

All Graphs

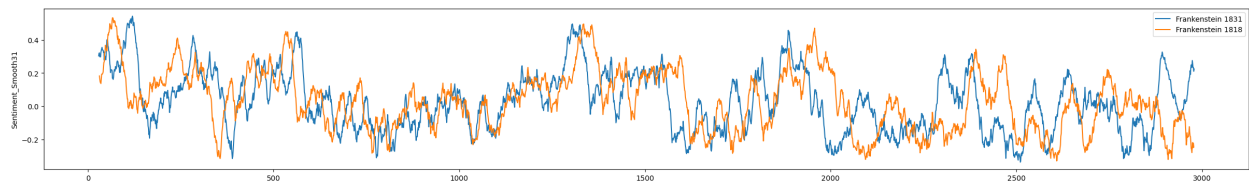
```
#| fig-cap: "A line plot indicating the sentiment scores of two Frankenstein editions over the course of the book by sentence length."
rough_graph
```

```
#| fig-cap: "A line plot indicating the sentiment scores of two Frankenstein editions over the course of the novel."
dropped_graph
```



```
#| fig-cap: "A line plot indicating the sentiment scores of two Frankenstein editions over the course of the novel."
imputed_graph
```



Discussion

Overall, both methods of balancing the datasets produced similar results. There are some differences in the peaks and valleys of the sentiment scores, but the overall trends are consistent across both methods. This suggests that our findings are robust to the method of balancing used. Additional indicators of reliability would be the near perfect matchup of both smoothed sentiment lines at around the 1000 mark on the imputed graph.

In my own opinion, the expansion of the 1818 text through imputation seems to have produced a better result than the random dropped graph. While accounting for the variance between comparisons, the imputed graph seems to have a closer alignment of peaks and valleys between the two texts.

Additionally the results have an interesting literary implication. In the 1818 text, the stronger peaks in sentiment tend to occur earlier than the 1831 peaks until around the halfway mark where it switches after syncing up near the 1000 mark. The patterns in the data may indicate that overall, both editions share the same sentiment patterns at roughly the same times, however, the degree to which sentiment differs in some areas are interesting to point out. Take the 1500 - 2000 range for instance, where the sentiment context is certainly present in both texts however the height of peaks vary more. The starkest difference to note is the ending where the contextual sentiment overlap splits dramatically. This may indicate that the changes made in the 1831 edition altered the tone of the ending significantly compared to the original 1818 text. This could be a point of further literary analysis to explore how the revisions impacted the overall sentiment and themes of the novel's conclusion.

This analysis demonstrates how NLP techniques can be applied to literary texts to uncover insights about authorial revisions and their effects on narrative tone. By quantitatively comparing sentiment across different editions, we can gain a deeper understanding of how changes in language reflect shifts in thematic emphasis

and emotional impact. This approach demonstrates the potential of computational methods to enhance the traditional study of literature.